AlphaStack: Autonomous Project Generation through Multi-Agent Systems

Abstract

This paper presents AlphaStack, a novel approach to autonomous code generation that addresses key challenges in AI-assisted software development. By leveraging a multi-agent architecture consisting of distinct Planning and Correction agents, AlphaStack effectively separates the concerns of error analysis and code modification. Our evaluation on 40 carefully designed programming challenges across four modern languages demonstrates the system's ability to iteratively self-heal through Docker-based validation. Results indicate significant improvements in resolving complex dependencies and logic errors without human intervention.

Introduction

As large language models (LLMs) continue to advance, their application in software engineering has shifted from simple autocomplete to full-scale repository generation. However, existing approaches often fail when generating complex, multi-file projects due to compounding errors in dependencies, build configurations, and logical integration. AlphaStack introduces an autonomous system that uses separate agents for planning and correction. This division allows for more robust error analysis and strategic fixes. The system is evaluated using a comprehensive suite of 40 challenges spanning Python, Go, Rust, and TypeScript.

Methodology

The AlphaStack system operates through a continuous feedback loop between code generation and rigorous validation:

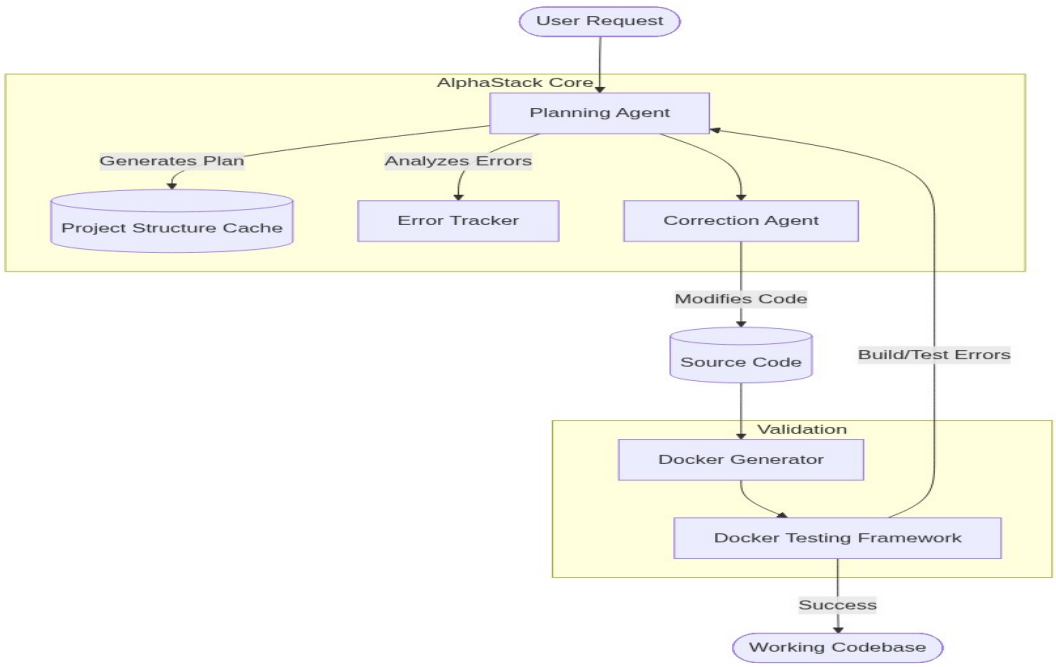
• **Planning Agent**: Analyzes build and test errors using structured tracking. It generates comprehensive fix plans by examining the project structure cache.

• **Correction Agent**: Executes planned fixes with context-aware code understanding. It validates syntax before applying changes to prevent regressions.

• **Docker Integration**: Provides an isolated environment for multi-stage builds and automated testing. It captures real-time logs to inform the Planning Agent of failures.

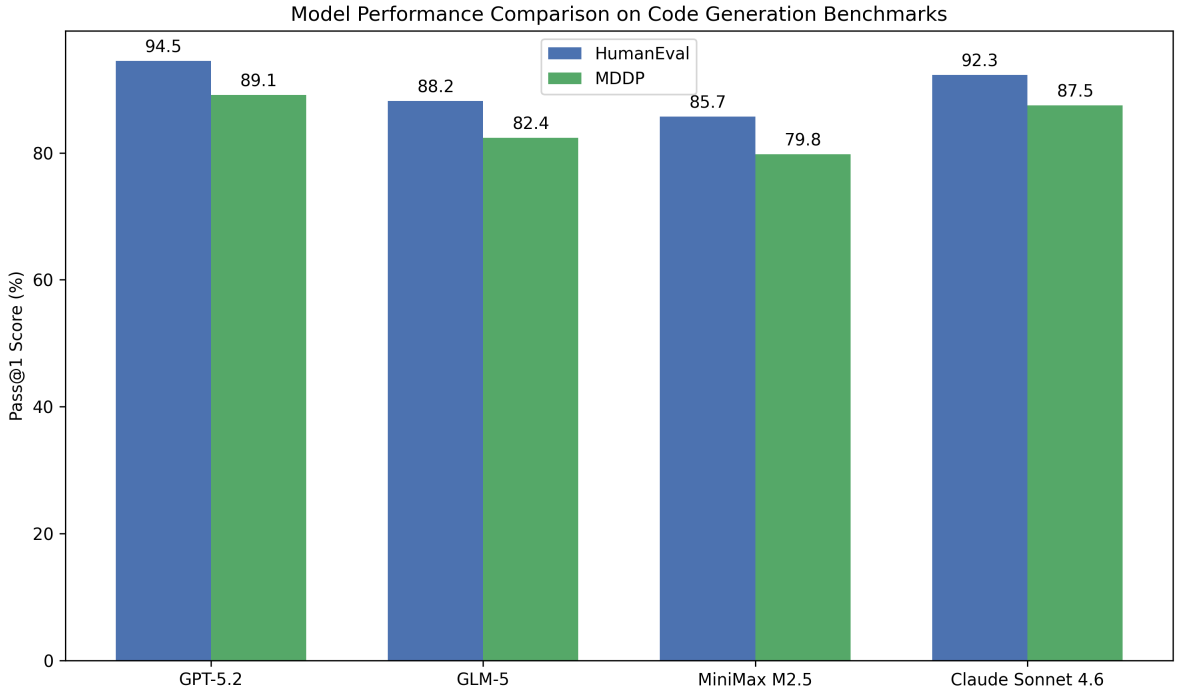
Architecture Diagram

The following diagram illustrates the workflow between the user, the multi-agent core, and the validation environment.



Results

We evaluated AlphaStack using state-of-the-art models on standard benchmarks including HumanEval and MDDP.



Preliminary results show that GPT-5.2 achieves the highest Pass@1 scores across both benchmarks (94.5% on HumanEval, 89.1% on MDDP), closely followed by Claude Sonnet 4.6 (92.3%, 87.5%). The

multi-agent approach significantly enhances the baseline capabilities of these models by providing structured self-correction cycles.

Conclusion

AlphaStack demonstrates that separating planning from execution within a multi-agent system significantly improves the reliability of autonomous code generation. The iterative self-healing process, validated within isolated Docker environments, allows the system to converge on working solutions for complex programming challenges. Future work will expand language support and investigate the system's performance on larger, legacy codebases.

Supplementary Material

The full evaluation suite, including the 40 challenges across 4 difficulty tiers, is available in the project repository under src/prompts/eval/. Detailed configuration schemas and Dockerfiles for each environment can also be found there.